RUGGED MONITORING

Enterprise Solution Architecture Document

Skill Matrix + Learning Roadmap + Career Progression Platform

Version 1.0 | Generated for Implementation in Cursor
2026-06-08

Document Purpose

This document converts the product concept into a production-ready engineering blueprint. It defines system structure, role-based access control, database architecture, APIs, frontend route hierarchy, analytics pipelines, AI recommendation engine, deployment model, and sprint-by-sprint Cursor implementation plan.

Table of Contents

1. Executive Summary

Rugged Monitoring is an enterprise workforce intelligence platform designed to connect employee skills, required role competencies, learning programs, assessments, certificates, and promotion readiness into one operational system. The platform is not a generic LMS. It is a skill-driven career progression engine.

- Primary goal: map every employee to their role, experience level, required skills, learning roadmap, assessment status, certificate status, and promotion readiness.
- Primary users: employee, instructor, team leader, department manager, HR manager, admin, and CEO.
- Primary intelligence: skill gaps, readiness scores, certification risk, training ROI, and role progression readiness.
- Primary admin capability: role-based sidebars and permission toggles controlled from Roles and Permissions.
- Primary deployment target: Next.js frontend, Node/NestJS backend, PostgreSQL, Prisma, Redis, S3, Sentry, and CI/CD.

Outcome	Description
Enterprise skill visibility	HR and leaders can see skill coverage by employee, team, department, role, and experience level.
Learning roadmap automation	Employees receive the next required 101, 201, 301, or 401 learning path based on current skill status.
Assessment validation	Skills are validated using tests, assignments, question banks, and score thresholds.
Certificate compliance	Certificates can be issued, expired, renewed, revoked, and verified using QR or public verification routes.
Promotion readiness	The system calculates readiness based on required skills, courses, assessment scores, certificates, and manager approval.

2. Business Domain and User Journey

The core journey is built around career growth rather than course consumption. Skills drive learning, learning drives assessments, assessments drive certification, and certificates contribute to promotion readiness.

Primary Business Flow

```
Employee joins company
  -> Assigned Department, Team, Role, Experience Level
  -> System loads required skills for that role
  -> Skill Matrix compares current score vs required score
  -> Gap Engine identifies missing 101/201/301/401 levels
  -> Learning Roadmap Engine assigns next courses
  -> Employee completes learning modules
  -> Assessment Engine validates skill level
  -> Certificate Engine issues certificate
  -> Promotion Readiness Engine calculates readiness
  -> Manager/HR reviews and approves next role progression
```

Stage	System Object	Decision Logic
Role assignment	User, Role, Department, Team	User receives base menu and feature access.
Career framework	JobRole, ExperienceLevel	Defines required skills and expected skill levels.
Skill analysis	SkillMatrix, EmployeeSkill	Calculates current level and gap.
Learning roadmap	LearningPath, Course	Unlocks next course based on prerequisites.
Assessment	AssessmentResult	Pass/fail based on score and attempt rules.
Certification	Certificate	Issued after valid completion and assessment pass.
Promotion readiness	PromotionReadiness	Aggregates all career requirements into one score.

3. Platform Scope and Core Modules

Module	Scope
Dashboard	Role-aware landing page with KPIs, tasks, and quick actions.
Employees	Directory, profiles, manager assignment, role history, skill state, certificates, learning progress.
Career Framework	Roles, experience bands, promotion rules, role skill requirements.
Skill Admin	Skill library, levels, categories, mappings, validity rules.
Skill Matrix	Employee vs skill heatmap, team matrix, department matrix, gap analysis.
Learning Programs	Roadmaps, assigned learning, self-learning, locked/unlocked course progression.
Course Admin	Course creation, modules, lessons, quizzes, prerequisites, role mapping.
Assessments	Question bank, tests, attempts, scoring, results, recommendations.
Certificates	Templates, issue, renew, revoke, QR verification, expiry tracking.
Analytics	Skill readiness, training ROI, learning completion, assessment trends.
Reports	PDF, CSV, Excel exports for employees, departments, skills, learning, compliance.
Settings	General, security, email, notifications, integrations,

	appearance, system controls.
Audit Logs	Every sensitive admin, role, permission, course, and certificate event is logged.

4. Role Hierarchy and RBAC Model

The platform uses both role-based access control and optional user-level permission overrides. A user receives access from role permissions, custom role permissions, and explicit user permission overrides. The sidebar is generated from the permission set, not hardcoded.

Role Hierarchy

```
CEO
  Admin
    HR Manager
      Department Manager
        Team Leader
          Instructor
            Employee
```

Role	Primary Access	Should Not Access
Employee	My dashboard, my skills, learning roadmap, assessments, certificates, profile.	Admin settings, all employees, global reports, permissions.
Instructor	Courses assigned to teach, course management, student progress, assessments.	Global settings, salary or executive reports.
Team Leader	Team members, team skill matrix, assign learning, team progress, team reports.	Company-wide HR controls, system settings.
Department Manager	Department employees, department matrix, learning programs, reports.	System security, global permissions.
HR Manager	Employees, departments, instructors, compliance reports, skill matrix, learning data.	Infrastructure settings unless enabled.
Admin	All operational and system administration including settings and RBAC.	CEO-only financial/executive notes if configured.
CEO	Executive dashboard, workforce readiness, AI insights, executive reports.	Technical settings, API keys, audit mutation actions.

Permission Area	Employee	TL	Dept Mgr	HR	Admin	CEO
Dashboard	Self	Team	Dept	All	All	Executive
Employees	Self	Team view	Dept view	All manage	All manage	View
Skill Matrix	Self	Team	Dept	All	All	View
Learning	Self	Assign team	Manage dept	Manage all	Full	View
Courses	View	Assign	Assign/manage	Manage	Full	View
Certificates	Self	Team	Dept	All	Full	View
Analytics	Self	Team	Dept	HR/global	Full	Executive
Settings	Profile	Profile	Profile	Limited	Full	Profile
Audit Logs	No	No	No	Limited	Full	View only

5. System Context Architecture

System Context

```
Users
  -> Web App (Next.js)
  -> API Gateway / BFF
  -> Auth + RBAC + Validation + Audit Middleware
  -> Domain Services
  -> PostgreSQL + Redis + S3 + Search Index
  -> Analytics Warehouse / Read Models
  -> Dashboards + Reports + AI Insight Engine
```

Layer	Responsibility	Technology
Presentation	Role-aware UI, dashboards, forms, tables, charts.	Next.js, Tailwind, Shadcn UI, React Query
BFF/API	Request validation, response shaping, auth checks, audit hooks.	NestJS or Next.js API routes
Domain Services	Users, roles, skills, learning, courses, assessments, certificates.	TypeScript services
Intelligence	Skill gap engine, roadmap engine, recommendation engine.	TypeScript jobs, optional Python/AI service later
Data	Transactional state and relations.	PostgreSQL, Prisma
Cache/Jobs	Fast dashboards, background work, scheduled reminders.	Redis, BullMQ
Storage	Videos, PDFs, certificates, exports.	S3-compatible storage
Observability	Errors, traces, logs, metrics.	Sentry, Pino, OpenTelemetry

6. Microservice Architecture

For first deployment, a modular monolith is recommended because it is faster, easier to deploy, and safer for Cursor implementation. The code should still be organized by domain service boundaries so it can be split into microservices later.

Service	Core Tables	Responsibilities
Identity Service	users, roles, permissions, sessions	Login, logout, tokens, RBAC, role sidebars.
Organization Service	departments, teams, job_roles	Company structure, reporting lines, team assignment.
Skill Service	skills, skill_categories, skill_levels, employee_skills	Skill library, skill levels, matrix, gap score.
Learning Service	courses, lessons, enrollments, learning_paths	Course admin, roadmaps, progress, assignments.
Assessment Service	assessments, questions, attempts, results	Tests, scoring, attempts, validation.
Certificate Service	certificates, certificate_templates	Issue, expire, revoke, QR verification.
Analytics Service	report_snapshots, analytics_cache	Aggregations, executive dashboard, exports.
Notification Service	notifications, email_templates	In-app, email, reminders, certificate expiry alerts.
Audit Service	audit_logs	Immutable record of sensitive actions.
Integration Service	integrations, webhooks	Slack, Teams, SSO, APIs, SCIM later.

7. Database ERD and Entity Design

Logical ERD

```
User -> Role -> RolePermission -> Permission
User -> Department
User -> Team
User -> EmployeeSkill -> Skill -> SkillCategory
Skill -> SkillLevel -> Course -> Assessment -> Certificate
JobRole -> RoleSkillRequirement -> SkillLevel
ExperienceLevel -> JobRole
LearningPath -> LearningPathStep -> Course
Course -> CourseModule -> Lesson
CourseEnrollment -> LessonProgress
Assessment -> AssessmentQuestion -> AssessmentAttempt -> AssessmentResult
Certificate -> CertificateTemplate
User -> Notification
User -> AuditLog
Settings -> FeatureFlags -> RoleSidebarItems
```

Entity	Important Fields	Notes
User	id, email, name, roleId, departmentId, teamId, managerId, status	Main employee record.
Role	id, name, hierarchyRank, isSystem	Defines default permission bundle.
Permission	id, key, module, action	Atomic permission like users.view.
Department	id, name, headId	Org grouping.
Team	id, departmentId, managerId	Team grouping.
JobRole	id, name, levelCode	Career ladder role like Intern, SDE1.
ExperienceLevel	id, label, minYears, maxYears	0-2, 2-5, etc.
Skill	id, name, categoryId, description	Core skill object.
SkillLevel	id, skillId, code, label, requiredScore	101 Basic, 201 Mid, 301 Advanced, 401 Expert.
EmployeeSkill	userId, skillId, currentLevelId, score	Current employee competency.
Course	id, skillId, levelId, title, type	Course linked to skill level.
Assessment	id, skillId, courseId, passingScore	Validation object.
Certificate	id, userId, skillId, courseId, expiresAt	Proof of completion.
PromotionRule	fromRoleId, toRoleId, requiredScore	Readiness requirements.
AuditLog	actorId, action, entityType, entityId	Compliance trace.
Setting	key, value, group	Platform configuration.

8. PostgreSQL Schema Blueprint

The following DDL is a production-style schema blueprint. Cursor should create migrations using Prisma or SQL migrations. All tables should include created_at and updated_at timestamps; sensitive events should also be copied to audit_logs.

Core SQL DDL

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TYPE user_status AS ENUM ('active','inactive','invited','suspended');
CREATE TYPE enrollment_status AS ENUM ('assigned','in_progress','completed','cancelled','expired');
CREATE TYPE certificate_status AS ENUM ('issued','expired','revoked','pending');
CREATE TYPE assessment_status AS ENUM ('draft','published','archived');

CREATE TABLE roles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(80) UNIQUE NOT NULL,
  slug VARCHAR(80) UNIQUE NOT NULL,
  hierarchy_rank INT NOT NULL DEFAULT 0,
  is_system BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE permissions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  permission_key VARCHAR(120) UNIQUE NOT NULL,
  module VARCHAR(80) NOT NULL,
  action VARCHAR(40) NOT NULL,
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE role_permissions (
  role_id UUID REFERENCES roles(id) ON DELETE CASCADE,
  permission_id UUID REFERENCES permissions(id) ON DELETE CASCADE,
  enabled BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE departments (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(120) UNIQUE NOT NULL,
  description TEXT,
  head_user_id UUID NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE teams (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  department_id UUID REFERENCES departments(id),
  name VARCHAR(120) NOT NULL,
  manager_user_id UUID NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(department_id, name)
);

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(180) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  first_name VARCHAR(80) NOT NULL,
  last_name VARCHAR(80) NOT NULL,
  role_id UUID REFERENCES roles(id),
  department_id UUID REFERENCES departments(id),
  team_id UUID REFERENCES teams(id),
  manager_id UUID REFERENCES users(id),
  job_title VARCHAR(120),
  years_experience NUMERIC(4,1) DEFAULT 0,
  status user_status NOT NULL DEFAULT 'invited',
  avatar_url TEXT,
```

```
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE departments ADD CONSTRAINT fk_department_head FOREIGN KEY (head_user_id) REFERENCES users(id);
ALTER TABLE teams ADD CONSTRAINT fk_team_manager FOREIGN KEY (manager_user_id) REFERENCES users(id);

CREATE TABLE user_permissions (
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  permission_id UUID REFERENCES permissions(id) ON DELETE CASCADE,
  allowed BOOLEAN NOT NULL,
  reason TEXT,
  PRIMARY KEY (user_id, permission_id)
);

CREATE TABLE skill_categories (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(120) UNIQUE NOT NULL,
  color VARCHAR(24),
  icon VARCHAR(60),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE skills (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  category_id UUID REFERENCES skill_categories(id),
  name VARCHAR(140) UNIQUE NOT NULL,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE skill_levels (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
  code VARCHAR(20) NOT NULL,
  label VARCHAR(80) NOT NULL,
  sequence INT NOT NULL,
  min_score INT NOT NULL DEFAULT 0,
  required_score INT NOT NULL DEFAULT 70,
  UNIQUE(skill_id, code)
);

CREATE TABLE employee_skills (
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
  current_level_id UUID REFERENCES skill_levels(id),
  score INT NOT NULL DEFAULT 0,
  source VARCHAR(80),
  verified_by UUID REFERENCES users(id),
  verified_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY(user_id, skill_id)
);

CREATE TABLE job_roles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(120) UNIQUE NOT NULL,
  code VARCHAR(40) UNIQUE NOT NULL,
  description TEXT,
  hierarchy_order INT NOT NULL
);

CREATE TABLE experience_levels (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  label VARCHAR(80) UNIQUE NOT NULL,
  min_years NUMERIC(4,1),
  max_years NUMERIC(4,1)
);

CREATE TABLE role_skill_requirements (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  job_role_id UUID REFERENCES job_roles(id) ON DELETE CASCADE,
  experience_level_id UUID REFERENCES experience_levels(id),
  skill_id UUID REFERENCES skills(id),
```

```
  required_level_id UUID REFERENCES skill_levels(id),
  weight NUMERIC(5,2) NOT NULL DEFAULT 1.0,
  is_mandatory BOOLEAN DEFAULT TRUE,
  UNIQUE(job_role_id, experience_level_id, skill_id)
);

CREATE TABLE courses (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  skill_id UUID REFERENCES skills(id),
  level_id UUID REFERENCES skill_levels(id),
  title VARCHAR(180) NOT NULL,
  description TEXT,
  duration_minutes INT DEFAULT 0,
  is_mandatory BOOLEAN DEFAULT FALSE,
  status VARCHAR(40) DEFAULT 'draft',
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE course_modules (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  course_id UUID REFERENCES courses(id) ON DELETE CASCADE,
  title VARCHAR(180) NOT NULL,
  sequence INT NOT NULL
);

CREATE TABLE lessons (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  module_id UUID REFERENCES course_modules(id) ON DELETE CASCADE,
  title VARCHAR(180) NOT NULL,
  lesson_type VARCHAR(40) NOT NULL,
  content_url TEXT,
  body TEXT,
  sequence INT NOT NULL
);

CREATE TABLE course_enrollments (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  course_id UUID REFERENCES courses(id) ON DELETE CASCADE,
  assigned_by UUID REFERENCES users(id),
  status enrollment_status DEFAULT 'assigned',
  progress_percent INT DEFAULT 0,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  due_at TIMESTAMPTZ,
  UNIQUE(user_id, course_id)
);

CREATE TABLE assessments (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  skill_id UUID REFERENCES skills(id),
  course_id UUID REFERENCES courses(id),
  title VARCHAR(180) NOT NULL,
  status assessment_status DEFAULT 'draft',
  passing_score INT NOT NULL DEFAULT 70,
  max_attempts INT DEFAULT 3,
  time_limit_minutes INT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE assessment_questions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  assessment_id UUID REFERENCES assessments(id) ON DELETE CASCADE,
  question_type VARCHAR(40) NOT NULL,
  prompt TEXT NOT NULL,
  options JSONB,
  correct_answer JSONB,
  points INT DEFAULT 1,
  sequence INT NOT NULL
);

CREATE TABLE assessment_attempts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  assessment_id UUID REFERENCES assessments(id),
```

```
  user_id UUID REFERENCES users(id),
  started_at TIMESTAMPTZ DEFAULT now(),
  submitted_at TIMESTAMPTZ,
  score INT,
  passed BOOLEAN,
  answers JSONB
);

CREATE TABLE certificate_templates (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(160) NOT NULL,
  html_template TEXT NOT NULL,
  is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE certificates (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  skill_id UUID REFERENCES skills(id),
  course_id UUID REFERENCES courses(id),
  assessment_attempt_id UUID REFERENCES assessment_attempts(id),
  template_id UUID REFERENCES certificate_templates(id),
  certificate_number VARCHAR(80) UNIQUE NOT NULL,
  status certificate_status DEFAULT 'issued',
  issued_at TIMESTAMPTZ DEFAULT now(),
  expires_at TIMESTAMPTZ,
  revoked_at TIMESTAMPTZ,
  verification_hash VARCHAR(180) UNIQUE NOT NULL
);

CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(180) NOT NULL,
  body TEXT,
  type VARCHAR(60),
  read_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  actor_id UUID REFERENCES users(id),
  action VARCHAR(120) NOT NULL,
  entity_type VARCHAR(80) NOT NULL,
  entity_id UUID,
  metadata JSONB,
  ip_address INET,
  user_agent TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_users_role ON users(role_id);
CREATE INDEX idx_users_department ON users(department_id);
CREATE INDEX idx_employee_skills_skill ON employee_skills(skill_id);
CREATE INDEX idx_courses_skill_level ON courses(skill_id, level_id);
CREATE INDEX idx_audit_logs_actor_created ON audit_logs(actor_id, created_at DESC);
CREATE INDEX idx_notifications_user_read ON notifications(user_id, read_at);
```

9. Prisma Model Blueprint

Prisma Schema Core Models

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum UserStatus {
  active
  inactive
  invited
  suspended
}

enum EnrollmentStatus {
  assigned
  in_progress
  completed
  cancelled
  expired
}

model Role {
  id            String           @id @default(uuid()) @db.Uuid
  name          String           @unique
  slug          String           @unique
  hierarchyRank Int              @default(0)
  isSystem      Boolean          @default(false)
  users         User[]
  permissions   RolePermission[]
  createdAt     DateTime         @default(now())
  updatedAt     DateTime         @updatedAt
}

model Permission {
  id            String           @id @default(uuid()) @db.Uuid
  permissionKey String           @unique
  module        String
  action        String
  description   String?
  roles         RolePermission[]
  users         UserPermission[]
  createdAt     DateTime         @default(now())
}

model RolePermission {
  roleId       String     @db.Uuid
  permissionId String     @db.Uuid
  enabled      Boolean    @default(true)
  role         Role       @relation(fields: [roleId], references: [id], onDelete: Cascade)
  permission   Permission @relation(fields: [permissionId], references: [id], onDelete: Cascade)
  @@id([roleId, permissionId])
}

model UserPermission {
  userId       String     @db.Uuid
  permissionId String     @db.Uuid
  allowed      Boolean
  reason       String?
  user         User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  permission   Permission @relation(fields: [permissionId], references: [id], onDelete: Cascade)
  @@id([userId, permissionId])
}

model Department {
  id          String   @id @default(uuid()) @db.Uuid
  name        String   @unique
  description String?
  headUserId  String?  @db.Uuid
  users       User[]
```

```
  teams       Team[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Team {
  id            String     @id @default(uuid()) @db.Uuid
  departmentId  String     @db.Uuid
  name          String
  managerUserId String?    @db.Uuid
  department    Department @relation(fields: [departmentId], references: [id])
  users         User[]
  createdAt     DateTime   @default(now())
  updatedAt     DateTime   @updatedAt
  @@unique([departmentId, name])
}

model User {
  id               String            @id @default(uuid()) @db.Uuid
  email            String            @unique
  passwordHash     String
  firstName        String
  lastName         String
  roleId           String?           @db.Uuid
  departmentId     String?           @db.Uuid
  teamId           String?           @db.Uuid
  managerId        String?           @db.Uuid
  jobTitle         String?
  yearsExperience  Decimal?          @default(0)
  status           UserStatus        @default(invited)
  avatarUrl        String?
  lastLoginAt      DateTime?
  role             Role?             @relation(fields: [roleId], references: [id])
  department       Department?       @relation(fields: [departmentId], references: [id])
  team             Team?             @relation(fields: [teamId], references: [id])
  manager          User?             @relation("Manager", fields: [managerId], references: [id])
  reports          User[]            @relation("Manager")
  employeeSkills   EmployeeSkill[]
  enrollments      CourseEnrollment[]
  attempts         AssessmentAttempt[]
  certificates     Certificate[]
  permissions      UserPermission[]
  notifications    Notification[]
  auditLogs        AuditLog[]         @relation("ActorAuditLogs")
  createdAt        DateTime           @default(now())
  updatedAt        DateTime           @updatedAt
}

model SkillCategory {
  id        String   @id @default(uuid()) @db.Uuid
  name      String   @unique
  color     String?
  icon      String?
  skills    Skill[]
  createdAt DateTime @default(now())
}

model Skill {
  id             String             @id @default(uuid()) @db.Uuid
  categoryId     String?            @db.Uuid
  name           String             @unique
  description    String?
  isActive       Boolean            @default(true)
  category       SkillCategory?     @relation(fields: [categoryId], references: [id])
  levels         SkillLevel[]
  employeeSkills EmployeeSkill[]
  courses        Course[]
  assessments    Assessment[]
  certificates   Certificate[]
  createdAt      DateTime           @default(now())
  updatedAt      DateTime           @updatedAt
}

model SkillLevel {
  id            String   @id @default(uuid()) @db.Uuid
  skillId       String   @db.Uuid
  code          String
```

```
  label         String
  sequence      Int
  minScore      Int      @default(0)
  requiredScore Int      @default(70)
  skill         Skill    @relation(fields: [skillId], references: [id], onDelete: Cascade)
  courses       Course[]
  @@unique([skillId, code])
}

model EmployeeSkill {
  userId         String      @db.Uuid
  skillId        String      @db.Uuid
  currentLevelId String?     @db.Uuid
  score          Int         @default(0)
  source         String?
  verifiedBy     String?     @db.Uuid
  verifiedAt     DateTime?
  user           User        @relation(fields: [userId], references: [id], onDelete: Cascade)
  skill          Skill       @relation(fields: [skillId], references: [id], onDelete: Cascade)
  currentLevel   SkillLevel? @relation(fields: [currentLevelId], references: [id])
  updatedAt      DateTime    @updatedAt
  @@id([userId, skillId])
}
```

Prisma Implementation Notes

```
// Continue the Prisma schema with Course, CourseModule, Lesson, CourseEnrollment,
// Assessment, AssessmentQuestion, AssessmentAttempt, CertificateTemplate,
// Certificate, Notification, AuditLog, Setting, Integration, and ReportSnapshot.
// Keep all model names singular, relation fields explicit, and use @@index for dashboard queries.
```

10. API Contract Design

All APIs must be protected by authentication except login, register, forgot password, reset password, public certificate verification, and health checks. Every write operation must create an audit log.

Method	Endpoint	Permission	Purpose
POST	/api/auth/login	Public	Login with email/password and receive access/refresh tokens.
POST	/api/auth/refresh	Public token	Refresh access token.
GET	/api/me	Authenticated	Return current user, permissions, sidebar.
GET	/api/users	users.view	Search, filter, sort, paginate users.
POST	/api/users	users.create	Create user and send invite.
PATCH	/api/users/:id	users.update	Update profile, role, team, department.
DELETE	/api/users/:id	users.delete	Soft-delete or deactivate user.
GET	/api/roles	roles.view	List roles with permission counts.
PATCH	/api/roles/:id/permissions	roles.manage	Update permission matrix.
GET	/api/departments	departments.view	List departments and analytics summary.
POST	/api/departments	departments.manage	Create department.
GET	/api/skills	skills.view	List skill library.
POST	/api/skills	skills.manage	Create skill with levels.
GET	/api/skill-matrix	skillmatrix.view	Return matrix by department, team, role, category.
POST	/api/courses	courses.manage	Create course and modules.
POST	/api/courses/:id/enroll	learning.assign	Assign or self-enroll.
POST	/api/assessments	assessments.manage	Create assessment.
POST	/api/assessments/:id/attempts	assessments.take	Start attempt.
POST	/api/assessments/:id/submit	assessments.take	Submit answers and compute score.
POST	/api/certificates/issue	certificates.issue	Issue certificate.
GET	/api/certificates/verify/:hash	Public	Verify certificate validity.
GET	/api/analytics/overview	analytics.view	Dashboard KPIs and trends.
GET	/api/reports/:type/export	reports.export	Export PDF/Excel/CSV.
GET	/api/audit-logs	auditlogs.view	Search audit events.
PATCH	/api/settings/:group	settings.manage	Update platform settings.

API Response Envelope

```
{
  "success": true,
  "data": {},
  "meta": {
    "page": 1,
    "pageSize": 20,
    "total": 120
  },
  "requestId": "req_abc123"
}
```

API Error Envelope

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
```

```
    "message": "You do not have permission to access this resource.",
    "details": {}
  },
  "requestId": "req_abc123"
}
```

11. Frontend Route and UI Hierarchy

Next.js Route Structure

```
app/
  (auth)/
    login/page.tsx
    forgot-password/page.tsx
    reset-password/page.tsx
  (platform)/
    layout.tsx
    dashboard/page.tsx
    employees/page.tsx
    employees/[id]/page.tsx
    career-framework/
      roles/page.tsx
      experience-levels/page.tsx
      promotion-rules/page.tsx
    skills/
      page.tsx
      categories/page.tsx
      [id]/page.tsx
    skill-matrix/page.tsx
    learning/
      page.tsx
      roadmaps/page.tsx
      courses/page.tsx
      courses/[id]/page.tsx
    assessments/page.tsx
    assessments/[id]/page.tsx
    certificates/page.tsx
    certificates/[id]/page.tsx
    analytics/page.tsx
    reports/page.tsx
    admin/
      users/page.tsx
      roles-permissions/page.tsx
      departments/page.tsx
      course-admin/page.tsx
      skill-admin/page.tsx
      categories/page.tsx
    settings/
      general/page.tsx
      security/page.tsx
      email/page.tsx
      notifications/page.tsx
      integrations/page.tsx
      appearance/page.tsx
      system/page.tsx
```

Screen	Primary Components	Data Sources
Dashboard	KPI cards, activity feed, chart grid, tasks	/api/analytics/overview, /api/me
Users	list/card toggle, role filter, department filter, user drawer	/api/users, /api/roles
Roles & Permissions	role selector, permission matrix, menu preview	/api/roles, /api/permissions
Skill Matrix	heatmap table, filters, export actions	/api/skill-matrix
Learning Roadmap	timeline, locked/unlocked courses, progress cards	/api/learning/roadmaps
Course Admin	course cards, builder, module editor	/api/courses
Assessments	question bank, assessment builder, result analytics	/api/assessments
Certificates	templates, issued list, expiry warnings	/api/certificates
Analytics	charts, drilldowns, gap insights	/api/analytics/*
Settings	tab navigation, settings forms, save/revert	/api/settings/*

12. Learning Roadmap Architecture

The learning roadmap is driven by skill levels and role requirements. Courses are not simply listed; they are sequenced and locked/unlocked based on prerequisite completion.

```
Java Roadmap
  101 Basic       Complete
  201 Midlevel    In Progress
  301 Advanced    Locked until 201 certificate issued
  401 Expert      Locked until 301 certificate issued

Roadmap state = Required Skill Level - Current Skill Level
Next recommended course = first incomplete unlocked course in sequence
```

Roadmap State	Trigger	System Action
Unlocked	Prerequisite course complete or no prerequisite	Employee can start course.
In Progress	Enrollment created and lessons started	Track progress and time spent.
Assessment Required	Course completed but assessment not passed	Show assessment CTA.
Certificate Pending	Assessment passed but certificate not issued	Queue certificate issue workflow.
Completed	Certificate issued and current skill updated	Unlock next level.
Expired	Certificate expired	Show renewal roadmap and alerts.

13. Skill Matrix Engine

The matrix must support employee, team, department, role, and category views. The score is computed from direct skill verification, assessment scores, course completion, certificate validity, and manager verification.

Scoring Logic

```
readinessScore = weightedAverage(
  employeeSkillScore,
  assessmentScore,
  certificateValidityScore,
  courseCompletionScore,
  managerVerificationScore
)

gapScore = max(requiredScore - readinessScore, 0)

promotionReadiness = weightedAverage(
  requiredSkillsCoverage,
  mandatoryCoursesCompletion,
  assessmentPassRate,
  validCertificatesRate,
  managerApprovalStatus
)
```

Score Range	Label	UI Color	Meaning
0-39	Critical Gap	Red	Immediate training required.
40-59	Needs Improvement	Orange	Learning path required.
60-79	Developing	Blue	Close to target, assessment may be needed.
80-100	Ready	Green	Meets role requirement.

14. Assessment and Certificate Engine

Capability	Implementation Detail
Question Bank	Reusable questions tagged by skill, level, difficulty, and topic.
Assessment Builder	Select questions manually or randomly by rule.
Attempts	Limit attempts, enforce cooldown, store answers and timing.
Scoring	Auto-score objective questions, manual review for essay.
Result Mapping	Passing assessment can update employee skill score and level.
Certificate Issue	Auto-issue after course completion and passing score.
QR Verification	Public verification hash route with certificate status.
Expiry Alerts	Notify employee, manager, and HR before expiry.
Revocation	Admin can revoke with reason, logged in audit logs.

```
Assessment Completion Flow
  Employee starts attempt
  -> Answers stored in draft state
  -> Submit validates time limit and attempt count
  -> Objective questions scored immediately
  -> Essay/manual review enters pending_review state
  -> Result determines pass/fail
  -> On pass, update EmployeeSkill and trigger Certificate issue
  -> On fail, recommend learning content and retake rules
```

15. Analytics Architecture

Analytics should use read-optimized aggregation services instead of heavy live joins for every dashboard request. Use scheduled jobs to compute daily snapshots and Redis for frequently accessed KPI data.

```
Transactional DB (PostgreSQL)
  -> Event/Audit Logs
  -> Aggregation Jobs
  -> analytics_snapshots / report_snapshots
  -> Redis dashboard cache
  -> UI charts and reports

Dashboard cache keys:
  analytics:overview:{role}:{scopeId}
  analytics:skill-readiness:{departmentId}
  analytics:cert-risk:{departmentId}
  analytics:learning-progress:{teamId}
```

Metric	Formula/Input	Consumer
Skill Readiness	Required skills met / required skills total	Managers, HR, CEO
Learning Completion	Completed enrollments / assigned enrollments	Team leads, HR
Certification Compliance	Valid certificates / required certificates	HR, Compliance, CEO
Promotion Readiness	Weighted readiness score by role rule	Managers, HR
Skill Gap Risk	Number of critical gaps by department	HR, CEO
Training ROI	Readiness improvement after training	HR, CEO
Assessment Quality	Pass rate, retake rate, average score	Instructors, Admin

16. AI Recommendation Engine

The AI engine should start as deterministic rules and evolve into LLM-assisted insights. Do not make the first version dependent on LLM output for core business logic. Use AI for explanations, recommendations, and summaries, while scores remain deterministic and auditable.

Engine	Input	Output
Skill Gap Engine	Role requirements, employee skills, assessments	Missing skills and priority order.
Learning Recommendation Engine	Skill gaps, course map, prerequisites	Next best courses and roadmap.
Promotion Readiness Engine	Skill coverage, certificates, assessments	Readiness score and blockers.
Certificate Risk Engine	Expiry dates, required certificates	Risk alerts and renewal queue.
AI Insight Generator	Aggregated analytics, trends	Plain-English executive summaries.
Workforce Forecast Engine	Trend snapshots	Forecast skills likely to become bottlenecks.

```
Example AI Insight Prompt Template
System: You are an enterprise workforce analytics assistant.
Input: Department skill readiness data, certification risk, learning completion trends.
Rules: Do not invent numbers. Only summarize provided metrics. Return blockers, opportunities, and recommended actions.
Output JSON:
{
  "summary": "...",
  "risks": ["..."],
  "recommendations": ["..."],
  "confidence": "high|medium|low"
}
```

17. Settings, Integrations, and Audit Logs

Settings Section	Configuration
General	Company name, logo, timezone, language, default role, fiscal year.
Security	Password policy, MFA, session timeout, login lockout, IP allowlist.
Email	SMTP provider, templates, sender identity, test email.
Notifications	In-app, email, reminders, weekly summaries, quiet hours.
Integrations	Slack, Teams, SSO, Google Workspace, SCIM, webhooks.
Appearance	Theme, brand colors, sidebar density, card style.
System	Maintenance mode, backups, cache clear, retention policies.

Audit Action	Trigger
auth.login	User successfully logs in.
user.role_changed	Role is changed.
permission.updated	Role permission matrix is modified.
course.updated	Course content or requirement changes.
certificate.revoked	Certificate revoked by admin.
settings.updated	Any system setting changes.
report.exported	User exports sensitive report.
integration.updated	API keys or integration settings changed.

18. Security Architecture

- Use httpOnly secure cookies for refresh tokens and short-lived access tokens for API calls.
- Hash passwords using Argon2 or bcrypt with strong cost parameters.
- Enforce RBAC at both API route and service method level.
- Never trust frontend permission checks alone.
- Use Zod validation on every request body and query parameter.
- Rate-limit auth endpoints and sensitive mutation endpoints.
- Log security-sensitive events to audit_logs.
- Use signed URLs for private file downloads from S3.
- Apply tenant/company isolation if multi-tenant support is added later.
- Use secret rotation and environment validation at startup.

Risk	Control
Unauthorized admin access	RBAC middleware, permission service, audit logs.
Token theft	httpOnly cookies, refresh rotation, short access token TTL.
SQL injection	Prisma parameterized queries, input validation.
XSS	React escaping, CSP headers, sanitized rich text.
Sensitive report leakage	Export permission checks, signed links, audit logs.
Privilege escalation	Role permission mutation restricted to Admin only and logged.
File abuse	S3 content-type restrictions, file size limits, virus scan later.

19. AWS Deployment Architecture

AWS Target Architecture

```
Route 53
  -> CloudFront
    -> Vercel / S3 static frontend OR Next.js hosted app
  -> API Gateway / ALB
    -> ECS Fargate service (NestJS API)
      -> RDS PostgreSQL
      -> ElastiCache Redis
      -> S3 private bucket
      -> SES/Resend email provider
      -> SQS for background jobs
      -> CloudWatch logs
      -> Sentry error tracking

Optional enterprise additions:
  - WAF for API protection
  - Secrets Manager for keys
  - OpenSearch for large audit/search workloads
  - EventBridge for scheduled expiry jobs
```

Component	AWS Option	MVP Alternative
Frontend	Amplify/CloudFront	Vercel
Backend	ECS Fargate	Railway/Render
Database	RDS PostgreSQL	Supabase/Postgres
Cache	ElastiCache Redis	Upstash Redis
Storage	S3	Supabase Storage
Email	SES	Resend
Jobs	SQS + ECS worker	BullMQ + Redis
Secrets	Secrets Manager	Platform env vars
Monitoring	CloudWatch + Sentry	Sentry only

20. CI/CD and Observability

```
GitHub Actions Pipeline
  1. Install dependencies
  2. Lint TypeScript
  3. Run unit tests
  4. Run Prisma validate
  5. Run integration tests
  6. Build frontend and backend
  7. Run database migration in staging
  8. Deploy staging
  9. Run smoke tests
  10. Manual approval for production
  11. Deploy production
  12. Run health checks
  13. Notify Slack/Teams
```

Signal	Tool	Purpose
Errors	Sentry	Frontend/backend exceptions.
Logs	Pino + CloudWatch	Structured request and job logs.
Metrics	OpenTelemetry	Latency, throughput, error rate.
Uptime	Health checks	API and database availability.
Audit	audit_logs table	Business/security event trail.
Product usage	PostHog	Feature adoption and user flow analysis.

21. Testing Strategy

Test Type	Scope	Examples
Unit	Pure functions and services	RBAC permission resolution, readiness score calculation.
Integration	API + DB	Create course, assign learning, complete assessment.
E2E	Browser user flows	Admin creates role; employee completes roadmap.
Security	Auth and permissions	Employee cannot access admin APIs.
Regression	Critical business flows	Certificate issuance after passing assessment.
Load	Dashboards and matrix	Skill matrix for 10,000 employees.
Accessibility	UI	Keyboard navigation, contrast, labels.
Export	Reports	PDF/Excel/CSV accuracy and permissions.

- Mock external providers in tests: email, S3, Slack, Teams.
- Use seed data for every role to test sidebar and RBAC behavior.
- Add snapshot tests for permission matrices and generated sidebar items.
- Add contract tests for every API envelope and error code.
- Add performance tests for analytics aggregations and skill matrix filters.

22. Sprint-by-Sprint Cursor Implementation Plan

Sprint	Theme	Deliverable
Sprint 1	Foundation	Create repo, Next.js app, NestJS/API layer, Prisma, PostgreSQL, env validation, Docker.
Sprint 2	Auth + RBAC	Login, refresh token, roles, permissions, guards, dynamic sidebar.
Sprint 3	Users + Org	Users, departments, teams, employee profile, role assignment.
Sprint 4	Career Framework	Job roles, experience levels, role skill requirements, promotion rules.
Sprint 5	Skill Admin	Skill categories, skills, skill levels, mappings, admin UI.
Sprint 6	Skill Matrix	Matrix API, heatmap UI, filters, readiness/gap scoring.
Sprint 7	Course Admin	Courses, modules, lessons, content uploads, course mapping.
Sprint 8	Learning Roadmaps	Roadmap engine, course assignments, locked/unlocked steps, progress.
Sprint 9	Assessments	Question bank, builder, attempts, scoring, results.
Sprint 10	Certificates	Templates, issue/revoke/renew, QR verification, expiry jobs.
Sprint 11	Analytics	Dashboard KPIs, department/team analytics, caching, snapshots.
Sprint 12	Reports	PDF/CSV/Excel exports, executive and compliance reports.
Sprint 13	Settings + Integrations	General/security/email/notifications/ integrations/appearance/system.
Sprint 14	Audit + Security	Audit logs, rate limiting, hardening, Sentry, security tests.
Sprint 15	Deployment	CI/CD, staging, production deploy, smoke tests, backup plan.
Sprint 16	Polish	UI refinement, accessibility, performance, documentation, launch checklist.

23. Cursor Master Prompts

Use these prompts one by one in Cursor. Do not run the next prompt until the previous sprint is tested and committed.

Prompt 1 - Foundation

```
Build the monorepo with Next.js 15, TypeScript, Tailwind, Shadcn UI, NestJS or API layer, Prisma, PostgreSQL, Docker, env validation, repository/service/controller structure, globa
```

Prompt 2 - Auth and RBAC

```
Implement JWT auth with refresh token rotation, roles, permissions, role_permissions, user_permission_overrides, permission guard middleware, dynamic sidebar generation, seed roles
```

Prompt 3 - Users and Organization

```
Implement user management with list/card views, filters, department/team CRUD, employee profile, manager assignment, status changes, audit logs, and tests.
```

Prompt 4 - Career Framework

```
Implement job roles, experience levels, role skill requirements, promotion rules, and promotion readiness calculation service.
```

Prompt 5 - Skills and Matrix

```
Implement skill categories, skills, 101/201/301/401 skill levels, employee skills, skill matrix APIs, heatmap UI, exports, and gap scoring.
```

Prompt 6 - Learning and Courses

```
Implement course admin, course modules, lessons, enrollments, learning roadmap engine, locked/unlocked course levels, progress tracking, and assigned learning.
```

Prompt 7 - Assessments and Certificates

```
Implement assessment builder, question bank, attempts, scoring, result history, certificate templates, issue/revoke/renew workflows, expiry jobs, and public verification route.
```

Prompt 8 - Analytics and Reports

```
Implement analytics aggregations, dashboard cards, chart APIs, report exports, executive views, caching, and scheduled snapshot jobs.
```

Prompt 9 - Settings and Integrations

```
Implement settings pages for General, Security, Email, Notifications, Integrations, Appearance, and System with validation, audit logs, and permission protection.
```

Prompt 10 - Production Hardening

```
Add rate limiting, CORS, Helmet, input sanitization, security tests, E2E tests, Sentry, Pino logs, GitHub Actions, deployment docs, backup plan, and production checklist.
```

24. Production Launch Checklist

Area	Checklist
Database	Migrations applied, indexes created, backups configured, rollback plan tested.
Auth	Refresh tokens secure, password hashing verified, MFA setting available.
RBAC	Every route protected, permission matrix seeded, sidebar matches permissions.
Data	Seed roles, sample skills, skill levels, demo users, departments, and courses.
Files	S3 bucket private, signed URLs used, upload limits configured.
Email	Templates tested for invite, reset, assignment, certificate, expiry.
Analytics	Snapshot jobs enabled, dashboard cache warming configured.
Audit	Sensitive actions logged and searchable.
Security	Rate limiting, CORS, Helmet, validation, XSS protections enabled.
Observability	Sentry DSN configured, logs structured, health checks live.
Testing	Unit, integration, E2E, permission, and smoke tests passing.
Deployment	Staging and production env vars separated, CI/CD green.
Documentation	Admin guide, user guide, API docs, deployment guide completed.
Launch	Go-live smoke test, backup snapshot, rollback package ready.

Appendix A - Role-Based Sidebar Menu Blueprint

```
Employee
  My Dashboard
  My Skills
  Skill Assessments
  Learning Roadmap
  Course Catalog
  My Certificates
  Goals
  Notifications
  Profile

Instructor
  Dashboard
  My Courses
  Course Management
  Student Progress
  Assessments
  Certificates
  Training Analytics
  Profile

Team Leader
  Dashboard
  Team Members
  Team Skill Matrix
  Assign Learning
  Team Progress
  Team Analytics
  Learning Requests
  Profile

Department Manager
  Dashboard
  Department Overview
  Department Employees
  Department Skill Matrix
  Learning Programs
  Certifications
  Reports
  Profile

HR Manager
  Dashboard
  Employees
  Instructors
  Departments
  Skill Matrix
  Assessments
  Learning
  Courses
  Categories
  Certificates
  Recruitment
  Compliance Reports
  HR Reports
  HR Analytics
  Profile

Admin
  Dashboard
  Skill Matrix
  Assessments
  Skills
  Learning
  Analytics
  Users
  Instructors
  Departments
  Roles & Permissions
  Course Admin
  Skill Admin
  Categories
  Certificates
  Reports
  Integrations
```

```
  Security
  Audit Logs
  Settings

CEO
  Executive Dashboard
  Workforce Overview
  Skill Readiness
  Learning Adoption
  Certification Compliance
  Executive Reports
  AI Insights
```

Appendix B - Permission Seed List

dashboard.view, users.view, users.create, users.update

users.delete, roles.view, roles.manage, permissions.manage

departments.view, departments.manage, teams.view, teams.manage

skills.view, skills.manage, skillmatrix.view, skillmatrix.export

career.view, career.manage, learning.view, learning.assign

learning.manage, courses.view, courses.manage, assessments.view

assessments.take, assessments.manage, certificates.view, certificates.issue

certificates.revoke, analytics.view, analytics.executive, reports.view

reports.export, settings.view, settings.manage, integrations.view

integrations.manage, auditlogs.view, notifications.view, notifications.manage

aiinsights.view

Appendix C - Detailed RBAC Implementation Rules

RBAC must be implemented as a server-side capability system. UI hiding is only convenience; the API must be the source of truth.

Rule	Implementation Requirement
Default deny	If no explicit permission is found, deny access.
Role grant	User receives permissions from assigned role where role_permissions.enabled = true.
User override allow	UserPermission.allowed = true can grant exception access.
User override deny	UserPermission.allowed = false must override role grants.
Sidebar filtering	Sidebar items are returned by /api/me/sidebar after permission resolution.
Scoped permissions	Manager scopes must restrict data to team or department even when screen is visible.
Audit mutation	Any role or permission change must write audit_logs with before/after metadata.
System roles	Employee, Instructor, TeamLeader, DepartmentManager, HRManager, Admin, CEO cannot be deleted.
Custom roles	Admin can create custom roles by cloning system role permissions.
Permission grouping	Group permissions by module in UI for clean toggles.

RBAC Pseudocode

```
type ResolvedPermission = {
  key: string;
  allowed: boolean;
  source: 'role' | 'user_override_allow' | 'user_override_deny';
  scope?: 'self' | 'team' | 'department' | 'all' | 'executive';
};

function can(user, permissionKey, resourceScope) {
  const resolved = resolvePermissions(user);
  if (!resolved[permissionKey]?.allowed) return false;
  return isScopeAllowed(user, resolved[permissionKey].scope, resourceScope);
}
```

Appendix D - Detailed API Contracts by Module

Module	Endpoint Group	Important Notes
Auth	/api/auth/*	Access tokens expire quickly; refresh token rotation required.
Me	/api/me, /api/me/sidebar	Returns user, role, permissions, sidebar, notification count.
Users	/api/users/*	Supports card/list views using same paginated query.
Departments	/api/departments/*	Department manager sees only assigned department unless elevated.
Skills	/api/skills/*	Skill levels generated automatically when creating a new skill if configured.
Skill Matrix	/api/skill-matrix/*	Use query params for scope, departmentId, teamId, roleId, categoryId.
Career	/api/career/*	Defines role progression and requirements.
Learning	/api/learning/*	Roadmaps are computed from required skills and employee skill state.
Courses	/api/courses/*	Course content uploads use signed URLs.
Assessments	/api/assessments/*	Attempts should store answers incrementally to avoid data loss.
Certificates	/api/certificates/*	Public verification endpoint must not expose private employee profile data.
Analytics	/api/analytics/*	Use cached aggregations and read models for dashboards.
Reports	/api/reports/*	Exports should be async for large reports.
Settings	/api/settings/*	Settings should validate group-specific schemas.
Audit	/api/audit-logs/*	Searchable by actor, action, entity, date range.

Example API Calls

```
GET /api/users?view=card&role=Instructor&departmentId=...&status=active&page=1&pageSize=12
GET /api/skill-matrix?scope=department&departmentId=...&categoryId=technical
GET /api/learning/roadmaps/me
POST /api/courses/:courseId/enrollments { userIds: [...], dueAt: "2026-07-01" }
POST /api/assessments/:id/attempts
POST /api/assessments/attempts/:attemptId/submit
GET /api/certificates/verify/:verificationHash
GET /api/analytics/executive?range=30d
POST /api/reports/export { type: "skill_gap", format: "xlsx", filters: {...} }
```

Appendix E - Frontend Component Inventory

Component	Purpose	Used In
RoleAwareLayout	Loads /api/me and renders correct sidebar.	All authenticated pages
SidebarNav	Nested menu with permission filtering.	Platform layout
PermissionGate	Hides/disables UI actions by permission.	Buttons, tabs, cards
DataTable	Reusable sortable, filterable table.	Users, courses, certificates
UserCardGrid	Modern employee card layout with role/status badges.	Users module
KpiCard	Metric card with icon, trend, sparkline.	Dashboards
HeatmapMatrix	Employee vs skill score grid.	Skill Matrix
RoadmapTimeline	101/201/301/401 progress timeline.	Learning Roadmap
CourseBuilder	Modules, lessons, quiz editor.	Course Admin
AssessmentBuilder	Question bank and test configuration.	Assessments
CertificatePreview	Template preview and QR verification preview.	Certificates
SettingsTabs	General/Security/Email/Notifications/ Integrations/Appearance/System.	Settings
AuditLogViewer	Timeline/table view with filters.	Audit Logs
ExportButton	PDF/Excel/CSV export with permission check.	Reports and matrix

Component Folder Suggestion

```
components/
  layout/RoleAwareLayout.tsx
  layout/SidebarNav.tsx
  auth/PermissionGate.tsx
  ui/KpiCard.tsx
  ui/DataTable.tsx
  users/UserCardGrid.tsx
  users/UserDrawer.tsx
  skills/HeatmapMatrix.tsx
  learning/RoadmapTimeline.tsx
  courses/CourseBuilder.tsx
  assessments/AssessmentBuilder.tsx
  certificates/CertificatePreview.tsx
  settings/SettingsTabs.tsx
  audit/AuditLogViewer.tsx
```

Appendix F - Data Seeding Plan

Seed Group	Records
Roles	Employee, Instructor, TeamLeader, DepartmentManager, HRManager, Admin, CEO.
Permissions	All module/action permissions from Appendix B.
Departments	Engineering, Product, HR, Operations, Compliance.
Teams	Frontend, Backend, DevOps, QA, Data, Security.
Skill Categories	Technical, Compliance, Tools, Internal Systems, Soft Skills, Cloud Computing.
Skills	Java, Python, Angular, React, AWS, Azure DevOps, Data Privacy, Bitbucket, Jira, Security.
Skill Levels	101 Basic, 201 Midlevel, 301 Advanced, 401 Expert for every skill.
Courses	One course per skill level for first 5 skills.
Assessments	One assessment per course and skill level.
Certificates	Templates for Course Completion, Skill Certification, Compliance Certificate.
Users	Demo users across all roles with different departments and skill states.
Settings	Default platform name, theme, security, email, notification settings.

```
Seed validation checklist:
- Admin user can see all sidebar items.
- Employee user sees only self-service menu items.
- HR can access employees but not security settings unless explicitly enabled.
- CEO sees executive dashboard and reports but not API keys or system settings.
- Skill Matrix renders realistic scores and missing values.
- Learning Roadmap shows complete, in-progress, and locked states.
```

Appendix G - Performance and Caching Strategy

- Use pagination for every list endpoint; never return all users or all matrix rows without limits.
- Use database indexes on user role, department, team, skill, course enrollment, and audit date fields.
- Cache dashboard KPI responses in Redis with short TTL and scope-based keys.
- Precompute analytics snapshots nightly and after major bulk operations.
- Use cursor pagination for audit logs and large report exports.
- Generate reports asynchronously when expected result size is large.
- Use signed S3 URLs for private documents rather than proxying large files through the API.
- Virtualize large skill matrix tables in the UI to avoid browser performance issues.

Area	Optimization
Users list	Pagination, search index, role/department indexes.
Skill Matrix	Server-side filters, virtualized table, cached department views.
Analytics	Snapshot tables, Redis cache, background aggregation jobs.
Reports	Async export queue, S3 result storage, notification on completion.
Course content	S3 + CDN, video streaming later.
Audit logs	Append-only table, date partitioning later for enterprise scale.

Appendix H - Migration and Release Strategy

Stage	Action
Local	Develop with Docker Compose PostgreSQL and seeded data.
Feature branch	Each Cursor sprint creates a branch and PR.
Staging migration	Run migrations automatically against staging.
Staging smoke	Run E2E flows for Admin, Employee, HR, Manager.
Production migration	Run migrations with backup snapshot first.
Feature flags	Enable risky modules gradually by role.
Rollback	Keep previous Docker image and migration rollback notes.
Post-release	Check Sentry, logs, health checks, and core workflows.

```
Release gates:
1. TypeScript build passes.
2. Prisma validate passes.
3. Unit and integration tests pass.
4. E2E smoke tests pass for each role.
5. RBAC route scan confirms no unprotected admin APIs.
6. Database backup created before migration.
7. Health checks pass after deployment.
```

Appendix I - Data Privacy and Compliance Notes

- Only collect data required for workforce learning, certification, and role readiness.
- Restrict employee analytics views by role and scope.
- Provide audit trails for administrative changes and sensitive exports.
- Do not expose private employee details on public certificate verification pages.
- Use data retention rules for audit logs, reports, and expired notifications.
- Encrypt data in transit with HTTPS and at rest using managed database/storage encryption.
- Use least privilege IAM roles for backend access to database, S3, and secrets.
- Allow admin to deactivate users without deleting historical certificates and audit records.

Appendix J - Final Cursor Execution Sequence

```
Recommended Cursor workflow:

1. Paste Prompt 1 and ask Cursor to implement foundation only.
2. Run npm install, prisma migrate dev, seed, and tests.
3. Commit working foundation.
4. Paste Prompt 2 for Auth/RBAC and dynamic sidebar.
5. Verify all seven demo roles render correct sidebar.
6. Commit.
7. Continue one module at a time.
8. After each prompt, run:
   - npm run lint
   - npm run typecheck
   - npm run test
   - npm run build
   - npx prisma validate
9. Do not accept code that bypasses permissions on the backend.
10. After Sprint 15, deploy staging before production.
```